

# FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

## Exam of WEDNESDAY 9 JULY 2025 – Laboratory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

### INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

### SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

**Theory** syntax and semantics of languages, divided in four sections: (75% of the final grade)

- |    |                                                  |                          |
|----|--------------------------------------------------|--------------------------|
| 1) | regular expressions and finite automata          | (1 exercise × 10 points) |
| 2) | syntax analysis and parsing methodologies        | (1 exercise × 10 points) |
| 3) | grammar design – translation – semantic analysis | (2 exercises × 5 points) |
| 4) | <i>bonus</i> question                            | (1 exercise × 5 points)  |

**Lab** compiler design by Flex and Bison (25% of the final grade)

- |    |                       |             |
|----|-----------------------|-------------|
| 1) | basic questions       | (30 points) |
| 2) | <i>bonus</i> question | (5 points)  |

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be  $\geq 18/30$  (*cum laude*  $\geq 32/30$ ).

## 1 Basic Questions (30 points)

1. Consider the implementation of the ACSE compiler shown in class.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the LANCE language with the ability to count the number of iterations performed in a while loop through a new **statement** called **`count_while`**.

The syntax of this new statement is as shown in the example below:

```
int i, n;

i = 256;
n = 777;
n = count_while (i != 1) {
    i = i / 2;
    write(n); // prints 777, 8 times
}
write(n); // prints 8
```

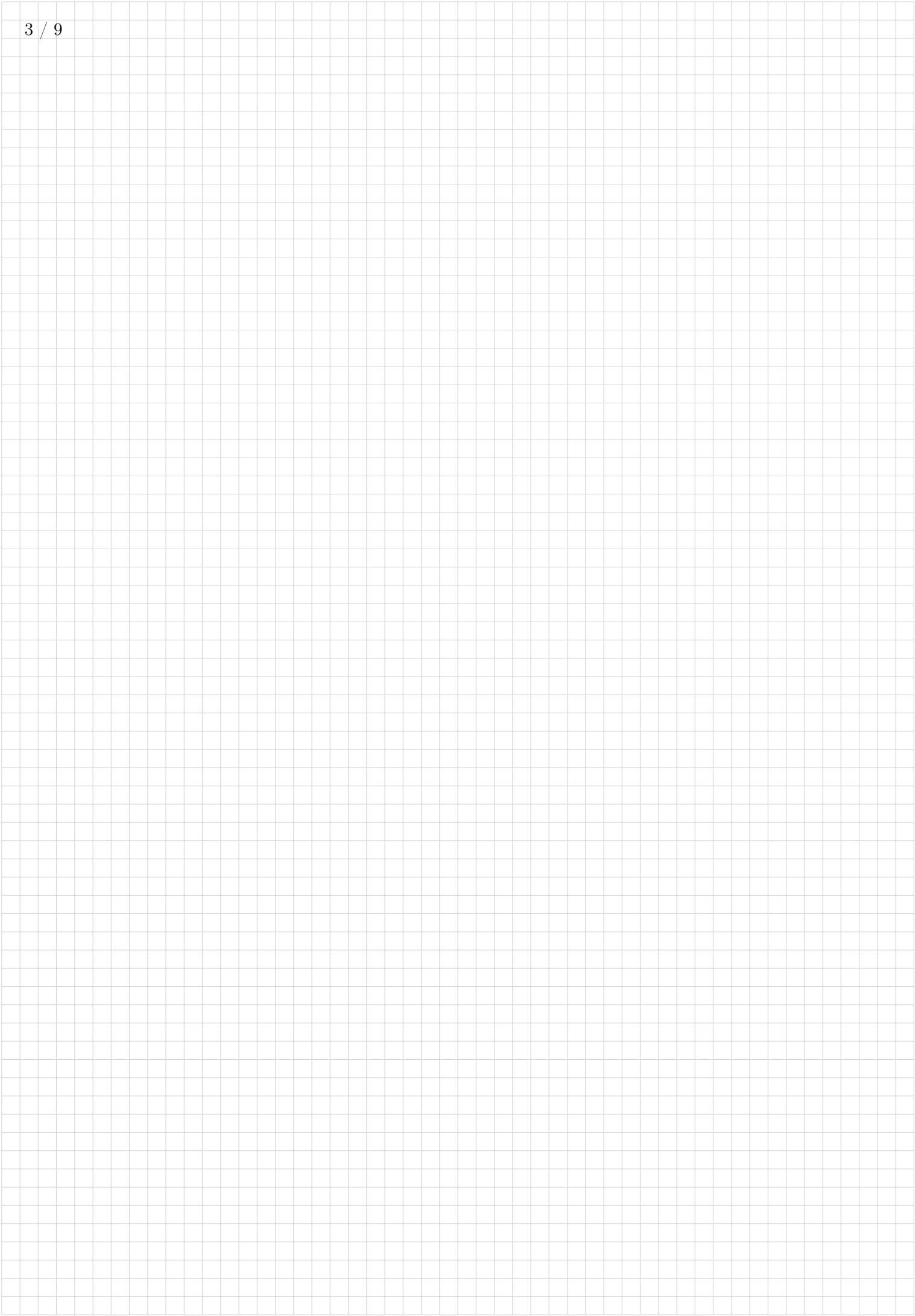
The `count_while` statement appears similar to a regular while statement but it is preceded by an assignment to a variable that — at the end of the execution of the loop — shall contain the number of iterations executed. In this example, the loop body executes 8 times, so the variable `n` has the value 8 at the end of the loop.

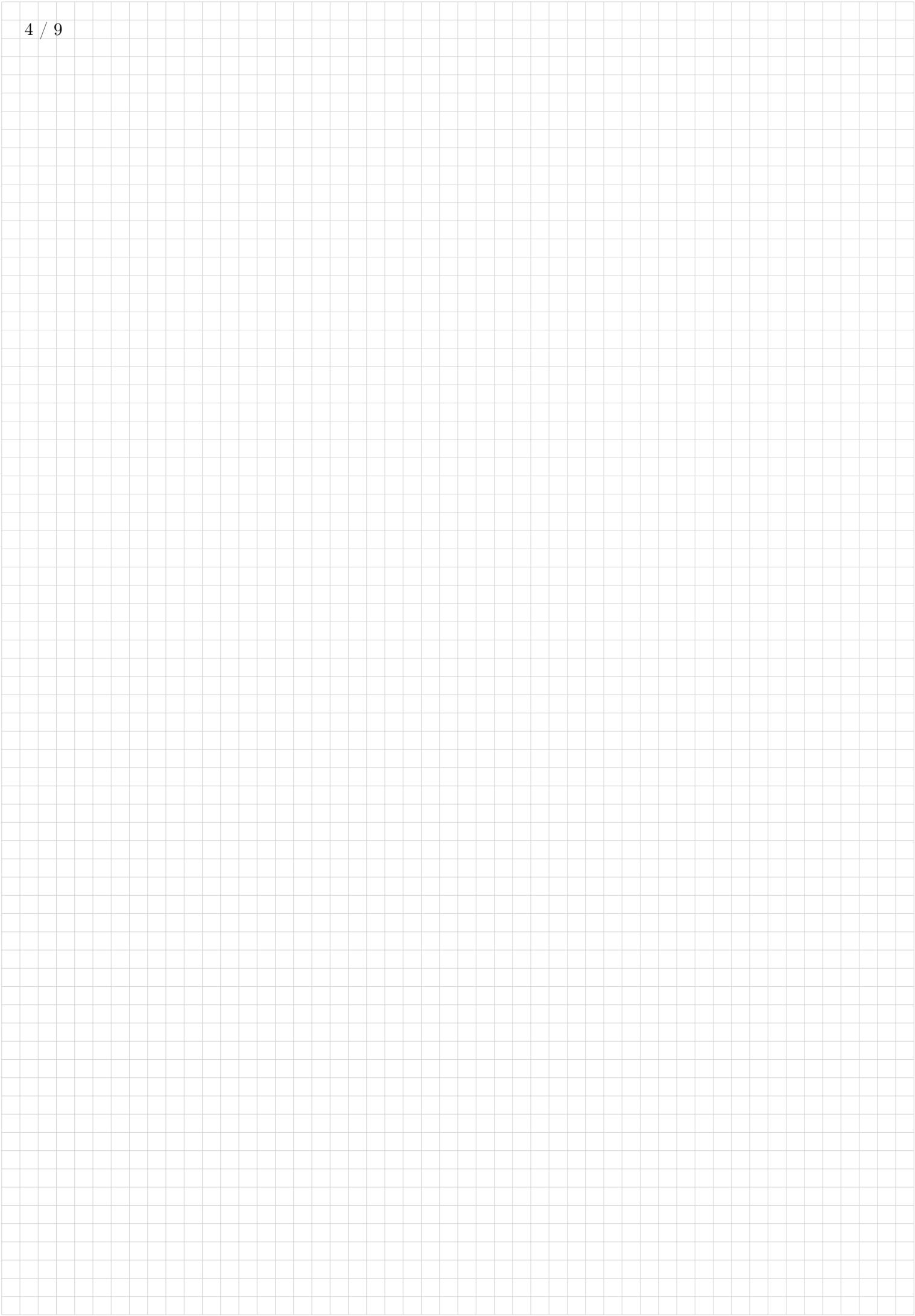
Additionally, your implementation must behave as follows:

- The `count_while` statement cannot appear inside a generic expression.
- The destination variable, loop condition and loop body are arbitrary variable identifiers, expressions, and blocks respectively.
- The parentheses around the expression are mandatory.
- Before the equals sign, only variables can appear (not arrays or array elements).
- The variable is only assigned once all the iterations of the loop are completed.

Answer the following questions:

- (a) Define the tokens (and the related declarations in **`scanner.l`** and **`parser.y`**). (1 points)
  - (b) Define the syntactic rules or the modifications required to the existing ones. (2 points)
  - (c) Define the semantic actions needed to implement the required functionality. (22 points)
-







2. Given the following LANCE code snippet:

```
--((craw))+(((craw)*craw)))
```

write down the syntactic tree generated during the parsing with the Bison grammar described in `parser.y` *starting from the `exp` nonterminal*. (5 points)

---



## 2 Bonus Question (5 points)

1. Show how to modify your previous implementation of the `count_while` statement in order to make it a valid expression operator. Write in detail which lines of code are changed and how. Does the example code previously shown remain syntactically valid with this modification? (2 points)
2. Consider the following snippets of ACSE compiler code, which define a new **st** statement:

### scanner.l

```
"st"          { return ST; }
```

### parser.y

```
%token ST

/* ... */

statement:
/* ... */
| st_statement SEMI
| SEMI
;

st_statement
: ST LPAR exp RPAR
{
    if ($3 == 123) {
        genADDI(program, $3, $3, 1);
    } else {
        genSUBI(program, $3, $3, 1);
    }
}
;
```

And the following LANCE program that uses the `st` statement defined as above:

### st.src

```
int x;
x = 100;
st(x + 23);
st(123);
```

How many ADDI and how many SUBI instructions are generated when compiling the two `st` statements, and why? Only consider the code specific to the `st` statement, and not its expression argument. (3 points)



